Worked Example: Silicon, kaldo vs. phonopy

May 9, 2026

This document maps the substrate's abstract states and operations onto the concrete materializations produced by kaldo and phonopy/phono3py for crystalline silicon, end to end from the Born–Oppenheimer potential through to the lattice thermal conductivity. It is the first contact between the architecture (`docs/symbolic_substrate.pdf`) and real physics. Equations are precise where the codes differ and sketchy where they do not. ShengBTE will be added as a third adapter when the cross-code machinery is ready for it.

1 Setup

System. Crystalline silicon, diamond structure, primitive cell with two atoms per unit cell.

Scope. Lattice thermal transport via the linearized Boltzmann transport equation, restricted to the harmonic approximation plus three-phonon scattering. Quantum statistics (f_{BE}). Quasi-harmonic Green–Kubo and four-phonon scattering deferred to later phases.

Reference codes.

- **kaldo** (Python). High-level API: `ForceConstants`, `Phonons`, `Conductivity`. Lazy evaluation; intermediate states exposed as attributes.
- **phonopy + phono3py** (Python+C). CLI-driven workflow, intermediate quantities in HDF5 files. Phonopy handles harmonic; phono3py handles anharmonic transport.

Conventions. Notation follows kaldo where it is unambiguous. Index conventions are not fixed at this stage; they become precise when adapters land.

Notation. V is the Born–Oppenheimer potential; $\{\mathbf{R}_i\}$ are atomic positions; u_i are displacements from equilibrium; M_α are atomic masses; $\mathbf{q}$ is a wavevector; ν is a band index.

2 Initial state: `Potential`

Abstract

The initial state is the Born–Oppenheimer potential energy surface,

$$V(\{\mathbf{R}_i\}) \;=\; V_0 \;+\; \sum_{n\geq 2} \frac{1}{n!} \sum_{i_1,\ldots,i_n} \Phi^{(n)}_{i_1\ldots i_n}\, u_{i_1} \cdots u_{i_n}, \tag{1}$$

treated as a continuum mathematical object (an analytic function of the atomic positions in a neighborhood of equilibrium).

Substrate type: `POTENTIAL` (parameter state, empty provenance).
Materialization: the externally-supplied potential. In Phase 1 the upstream is stubbed: the materialization is an opaque label identifying the choice of potential, not a modeled object.

Materialization in kaldo

kaldo does not represent the potential as a first-class object. The user supplies the potential by handing kaldo a *folder of pre-computed force constants* (see next section); the potential itself is implicit, accessed only through its second and third derivatives.

For the Tersoff path (`kaldo-examples/empirical_potentials/silicon_clathrate_Tersoff_LAMMPS`), the potential is realized as a LAMMPS pair-style declaration in `in.Si46`:

```
pair_style    tersoff
pair_coeff    * * forcefields/Si.tersoff Si
```

The materialization label is therefore `tersoff(Si.tersoff)`. The potential is callable (LAMMPS evaluates forces on demand) but not symbolically inspectable.

Materialization in phonopy / phono3py

Phonopy/phono3py likewise does not represent the potential as a first-class object. The user supplies displacements (generated by `phonopy -d` or `phono3py -d`) and externally runs a force calculator (VASP, QE, LAMMPS) on the displaced supercells. The potential is implicit in whichever code computed those forces.

For the canonical phono3py silicon example (`phono3py/example/Si-PBE`), the potential is QE/PBE with norm-conserving pseudopotentials. The materialization label is `dft(QE, PBE, Si.UPF, ecut=...)`. As with kaldo, the potential is callable through the external code but not symbolically inspectable.

Architectural note. Both adapters expose `Potential` as an opaque label upstream of the first computed state. This validates Phase 1's stubbed-upstream choice. Neither code lets us extract the Taylor coefficients of V symbolically; both require numerical sampling. The real architectural strain shows up at the next state.

3 `ForceConstants[order=2]`

Abstract

The harmonic force constants are the second derivative of the potential at equilibrium,

$$\Phi^{(2)}_{i\alpha,\,j\beta} \;=\; \left.\frac{\partial^2 V}{\partial u_{i\alpha}\,\partial u_{j\beta}}\right|_0, \tag{2}$$

indexed by atom pairs (i, j) and Cartesian components (α, β), evaluated on a finite supercell.

Substrate type: `FORCE_CONSTANTS`, with type-level parameter `order=2`.
Provenance (Phase 1, stubbed): a single operation `extract_derivative_and_truncate(Potential, order=2, method=`M`)` producing this state. The parameter $M \in \{$`finite_difference_lammps`, `dfpt`,

`finite_difference_qe`, `autodiff`, ...} is part of the operation's identity; different M produce different states (same type, different provenance).
Approximation error: truncation of the Taylor expansion at order 2 introduces $O(u^3)$ error in the energy and $O(u^2)$ error in derived spectral quantities; these errors attach to the operation, not to this state.
Discretization (lives on the materialization): supercell $N_1 \times N_2 \times N_3$, finite-difference displacement step δ (when M is a finite-difference method).

Materialization in kaldo

kaldo loads pre-computed force constants from a folder:

```
forceconstants = ForceConstants.from_folder(
    folder='fc_Si46',
    supercell=(3, 3, 3),
    format='lammps',
)
```

Format-specific files (Tersoff path): `Dyn.form` (binary, LAMMPS PHONON output) plus `replicated_atoms.xyz`.
Internal storage: `forceconstants.second` is a `SecondOrder` object wrapping a numpy array of shape $(n_{\text{rep atoms}}, 3, n_{\text{rep atoms}}, 3)$.
Underlying method (M): for the Tersoff example, M = `finite_difference_lammps`. LAMMPS's `dynamical_matrix` command computes $\Phi^{(2)}$ by central-differencing forces with displacement $\delta = 10^{-5}$ Å (`eskm 0.00001` in `in.Si46`).

Materialization in phonopy / phono3py

phonopy generates displacements, the user runs an external force calculator, and phonopy assembles the result into `fc2.hdf5`:

```
phonopy -d --dim 2 2 2 -c POSCAR-unitcell --pa auto   # generate displacements
# (run VASP/QE/LAMMPS on each disp-XXXXX/POSCAR)
phonopy -f disp-{00001..NNNN}/vasprun.xml             # collect forces
phonopy --writefc                                     # produce fc2.hdf5
```

File: `fc2.hdf5` (HDF5 with named datasets).
Internal storage: numpy array of shape $(n_{\text{atoms super}}, n_{\text{atoms super}}, 3, 3)$ accessible as `phonopy.force_constants`.
Underlying method (M): typically `finite_difference_`⟨`ext_code`⟩ with phonopy's symmetry-reduced displacement set; the displacement magnitude defaults to `0.01` Å (overridable via `--amplitude`).

Architectural strain (worth flagging). The abstract framing "$\Phi^{(2)} = \partial^2 V / \partial u^2$" suggests symbolic differentiation, but neither code performs it. Both numerically sample V at displaced configurations and apply finite-difference rules. This is the Tersoff/Taylor mismatch flagged in the planning discussion.

The substrate handles this honestly because *the method of derivative extraction is a parameter on the operation's identity.* The same abstract `ForceConstants[order=2]` type admits multiple operations:

- `extract_derivative_and_truncate(method=finite_difference_lammps,` δ=10^{-5}`)`

- `extract_derivative_and_truncate(method=finite_difference_qe,` δ=10^{-2}`)`
- `extract_derivative_and_truncate(method=dfpt,` ϵ_{conv}=...`)`
- `extract_derivative_and_truncate(method=autodiff)`

Each is a distinct operation (parameterized identity, §3.5.1 of the design doc); each produces a state with the same physics type but different provenance. "Same physics object, computed differently" becomes a structural distinction in the substrate, not a metadata tag.

4 ForceConstants[order=3]

Abstract

The cubic force constants are the third derivative,

$$\Phi^{(3)}_{i\alpha,\,j\beta,\,k\gamma} \;=\; \left.\frac{\partial^3 V}{\partial u_{i\alpha}\,\partial u_{j\beta}\,\partial u_{k\gamma}}\right|_0 . \tag{3}$$

Substrate type: `FORCE_CONSTANTS`, type-level parameter `order=3`.
Provenance: `extract_derivative_and_truncate(Potential, order=3, method=`M'`)`. The method M' for third-order is typically *different* from the second-order method: DFPT does not extend cleanly to third order, so M' is usually finite difference even when M is DFPT.
Approximation error: truncation at order 3 introduces $O(u^4)$ error (four-phonon and higher scattering, anharmonic renormalization, etc.).
Discretization: a third-order supercell $N'_1{\times}N'_2{\times}N'_3$, displacement step δ'. Crucially, $N' \neq N$ in general — the third-order supercell is typically smaller than the second-order one because the cost scales worse.

Materialization in kaldo

```
forceconstants = ForceConstants.from_folder(
    folder='fc_DFT',
    supercell=(8, 8, 8),       # 2nd-order supercell
    third_supercell=(3, 3, 3),  # 3rd-order supercell
    format='qe-sheng',
)
```

Format-specific files: `FORCE_CONSTANTS_3RD` (Sheng-style plain text) for QE path; `THIRD` (binary) for the LAMMPS path.
Internal storage: `forceconstants.third` is a `ThirdOrder` object; internal layout uses the `thirdorder.py` sparse convention by default.
Underlying method (M'): finite-difference of forces, supercell typically 3×3×3 for affordability.

Materialization in phonopy / phono3py

phono3py mirrors the phonopy workflow at third order:

```
phono3py -d --dim 2 2 2 -c POSCAR-unitcell          # 3rd-order displacements
phono3py --cf3 disp-{00001..NNNNN}/vasprun.xml      # collect forces
phono3py-load                                        # produce fc3.hdf5
```

File: `fc3.hdf5`.

Internal storage: numpy array of shape $(n_{\text{atoms super}},)^3 \times (3,3,3)$ accessible via `Phono3py.fc3` (compact form available too).
Underlying method (M'): finite-difference, displacement amplitude default `0.03` Å for third order (larger than the second-order default to improve signal relative to higher-order noise).

Note on supercell asymmetry. Both codes accept different supercells for second and third order. In the substrate, this means `ForceConstants[order=2]` and `ForceConstants[order=3]` are independent derived states with independent discretizations on their materializations. They share the same `Potential` as an upstream parameter but are otherwise unrelated by the abstract DAG. Good test of the type-level `order` parameter doing real work.

5 Correction operation: `enforce_acoustic_sum_rule`

Abstract

A correction operation that takes a force-constant tensor and returns a corrected one with exact translational invariance enforced:

$$\tilde{\Phi}^{(2)}_{i\alpha,\,j\beta}(R) \;\text{ such that }\; \sum_R \tilde{\Phi}^{(2)}_{i\alpha,\,j\beta}(R) \;=\; 0 \quad \forall\, i,\alpha,\beta. \tag{4}$$

This is Newton's third law in real space: the crystal's energy is invariant under uniform translation, so the rows and columns of the FC tensor must each sum to zero. Numerical FCs from finite-difference (or, less severely, from DFPT) violate this approximately; uncorrected, the violation appears as small spurious imaginary frequencies near $\mathbf{q} = 0$.

Substrate type signature: `enforce_acoustic_sum_rule` : `FC[order=`n`]` $\to$ `FC[order=`n`]`.
Operation parameters:
`variant` $\in$ {`simple_subtract`, `iterative_lstsq`, `born_huang`}.
Approximation error: $\epsilon_{\mathcal{O}} = 0$ in the abstract world (the operation is exact on the abstract object). The correction's effect on the materialization is bounded by $|\sum_R \Phi(R)|$ before enforcement, which is itself a discretization-error indicator for the input FC tensor.
Provenance: the input and output have the same physics type `FORCE_CONSTANTS` with the same `order`, but the output's provenance is the input's extended by this operation. They are formally distinct states (Decision 5) and downstream operations consume the ASR-enforced output, not the raw input.

This is the first "correction operation" in the worked example. Its shape—input and output share the same physics type, distinguished only by an additional provenance entry—is the template for all corrections (LO–TO splitting on `DynamicalMatrix`, permutation symmetrization on `FORCE_CONSTANTS[order=3]`, etc.).

Materialization in kaldo

kaldo enforces ASR at FC load via a flag on `from_folder`:

```
forceconstants = ForceConstants.from_folder(
    ...
    is_acoustic_sum=True,    # <-- triggers ASR
)
```

Internally, kaldo applies a simple residual-subtraction variant: for each (i, α, j, β), the residual $\sum_R \Phi^{(2)}(R)$ is divided over the on-site terms ($R = 0$) so that the new sum is zero exactly.
Variant: `simple_subtract`.
Timing: at FC load (before any q-space computation).
Default: *off*. The user must opt in.

Materialization in phonopy / phono3py

phonopy enforces ASR via the `--fc-symmetry` flag, applied during `phonopy --writefc`:

```
phonopy --fc-symmetry --writefc
```

The implementation iteratively projects out residual translations and applies permutation symmetry $(i, \alpha) \leftrightarrow (j, \beta)$, looping until both invariances hold simultaneously within a tolerance.
Variant: `iterative_lstsq` (combined with permutation symmetrization, which we treat as a separate operation).
Timing: at FC write (before `fc2.hdf5` is produced).
Default: on (when `--writefc` is invoked) for current versions; off in older phonopy.

Cross-code structural observation. kaldo and phonopy apply ASR *at different points in their internal pipelines* (load vs. write) and use *different variants* (simple subtraction vs. iterative). In the substrate, both adapters declare the same abstract operation `enforce_acoustic_sum_rule`, with the variant carried as an operation parameter. The internal-pipeline timing is invisible to the substrate; what the substrate sees is that both adapters' FC materializations downstream of `enforce_asr` can be aligned. The variant parameter makes the methodological difference visible without forcing the substrate to know where, internally, the codes apply it.

Note on order=3. Acoustic-sum-rule-like enforcement on third-order FCs (more properly, permutation symmetrization plus the order=3 acoustic invariance) is a separate correction operation in the substrate, with the same shape: `enforce_acoustic_sum_rule : FC[order=3]` $\to$ `FC[order=3]`. We do not detail it here because kaldo applies it implicitly via `thirdorder.py`'s convention, while phono3py applies it during `fc3.hdf5` production. The cross-code variant comparison follows the same pattern as for order=2.

6 DynamicalMatrix

Abstract

The dynamical matrix is the mass-weighted Fourier transform of the harmonic force constants,

$$D^{ij}_{\alpha\beta}(\mathbf{q}) = \frac{1}{\sqrt{M_i M_j}} \sum_{\mathbf{R}} \Phi^{(2)}_{i\alpha,\, j\beta}(\mathbf{R})\, e^{\,i\mathbf{q}\cdot\mathbf{R}}. \tag{5}$$

Substrate type: `DYNAMICAL_MATRIX`.
Operation: `compute_dynamical_matrix( ForceConstants[order=2] )`, structural operation, no approximation, $\epsilon_{\mathcal{O}} = 0$. The expected input is the ASR-enforced `ForceConstants[order=2]` produced by `enforce_acoustic_sum_rule`; consuming the raw FC tensor is structurally legal but produces a dynamical matrix with non-zero

acoustic-mode frequencies at $\mathbf{q} = 0$, which downstream operations may flag.
Discretization: q-mesh $K_1 \times K_2 \times K_3$ on which $D(\mathbf{q})$ is sampled.

Sketchy on purpose. kaldo and phonopy do not disagree on what the dynamical matrix is. The only divergence at this stage is whether LO–TO splitting has been added (relevant for polar materials, irrelevant for silicon). A separate operation `apply_lo_to_correction` would handle this where it matters; we omit it here for silicon.

Materialization in kaldo

Computed lazily on access:

```
phonons = Phonons(forceconstants=fc, kpts=[14, 14, 14], temperature=300, ...)
phonons.dynmat                          # triggers computation
```

Internal Cython kernel: `kaldo/observables/harmonic_with_q.py`.

Materialization in phonopy / phono3py

```
phonopy.run_band_structure(paths=...)        # or
phonopy.dynamical_matrix.set_dynamical_matrix(q)
phonopy.dynamical_matrix.dynamical_matrix    # 2x2 block per atom pair
```

Implementation: `phonopy/harmonic/dynamical_matrix.py`.

7 Dispersion

Abstract

The dispersion is the eigenstructure of the dynamical matrix,

$$D(\mathbf{q})\,\mathbf{e}_{\mathbf{q}\nu} \;=\; \omega^2_{\mathbf{q}\nu}\,\mathbf{e}_{\mathbf{q}\nu}, \qquad \omega_{\mathbf{q}\nu} \geq 0,\; \mathbf{e}\cdot\mathbf{e}^* = 1. \tag{6}$$

The state carries the mode frequencies $\omega_{\mathbf{q}\nu}$, the polarization vectors $\mathbf{e}_{\mathbf{q}\nu}$, and the group velocities $\mathbf{v}_{\mathbf{q}\nu} = \nabla_{\mathbf{q}}\,\omega_{\mathbf{q}\nu}$ (derived from Hellmann–Feynman or finite-differencing on the q-mesh).

Substrate type: `DISPERSION`.
Operation: `compute_dispersion(DynamicalMatrix)`, structural operation, $\epsilon_{\mathcal{O}} = 0$.
Discretization: same q-mesh as `DynamicalMatrix`.

Sketchy on purpose. Hermitian eigendecomposition is unambiguous. Group-velocity extraction has a small implementation choice (analytic via Hellmann–Feynman vs. finite-difference on the mesh) but the difference is negligible for silicon at standard mesh densities.

Materialization in kaldo

```
phonons.frequency       # shape (n_q, n_modes), units THz
phonons.eigenvectors    # shape (n_q, n_modes, n_modes)
phonons.velocity        # group velocities, shape (n_q, n_modes, 3)
```

Frequencies are stored in THz internally. Group velocities computed via Hellmann–Feynman inside `harmonic_with_q.py`.

Materialization in phonopy / phono3py

```
phonopy.set_band_structure(paths=...)
phonopy.get_band_structure_dict()      # returns {'frequencies', 'eigenvectors', 'group_v
```

Frequencies in THz. Group velocities via the `group_velocity` module (Hellmann–Feynman).

8 `ScatteringRates` and a cross-code σ-normalization finding

Abstract

The three-phonon scattering rate (imaginary self-energy, equivalently linewidth) for mode $(\mathbf{q}, \nu)$ is, schematically,

$$\Gamma_{\mathbf{q}\nu} \;=\; \frac{\pi}{N\hbar^2} \sum_{\mathbf{q}'\nu',\mathbf{q}''\nu''} \left|V^{(3)}\right|^2 \left[\,(n'+n''+1)\,\delta(\omega-\omega'-\omega'') + 2(n'-n'')\,\delta(\omega+\omega'-\omega'')\,\right], \quad (7)$$

with the energy-conservation δ-functions replaced numerically by a smooth kernel of width σ (Gaussian, Lorentzian, tetrahedron, ...).

Substrate type: `SCATTERING_RATES`, parameterized by `order=3` (three-phonon channel) and an explicit `scattering_processes` list.
Operation: `compute_scattering_rates`.
Operation parameters (physics-affecting, hence part of operation identity):
`broadening_kind` $\in$ {`gauss`, `lorentz`, `tetrahedron`, `adaptive_gauss`},
`sigma_THz` (numerical width if `broadening_kind` requires it),
`scattering_processes` $\subseteq$ {`3-phonon`, `isotopic`, `boundary`, ...}.
Approximation error: the broadening kernel introduces an $O(\sigma)$ deviation from the true δ-function in the limit of perfect $\mathbf{q}$-grid sampling; under finite mesh, finite σ is required and the error becomes a non-trivial function of both.

Materialization in kaldo

```
phonons = Phonons(..., third_bandwidth=sigma, broadening_shape='gauss')
phonons.bandwidth      # the linewidth Gamma_q, shape (n_q, n_modes)
```

Gaussian kernel (`kaldo/controllers/dirac_kernel.py`):

$$g_{\text{kaldo}}(x,\sigma) \;=\; \frac{1}{\sigma\sqrt{\pi}} \exp\!\left(-\frac{x^2}{\sigma^2}\right). \quad (8)$$

Standard deviation of g_{kaldo}: $\sigma/\sqrt{2}$.
Default: `third_bandwidth=None` $\Rightarrow$ adaptive bandwidth from the $\mathbf{q}$-mesh.

Materialization in phonopy / phono3py

```
ph3.sigmas = [sigma]
ph3.run_thermal_conductivity(...)
ph3.thermal_conductivity   # exposes Gamma internally during BTE solve
```

Gaussian kernel (`phono3py/phonon/func.py`):

$$g_{\mathrm{ph3}}(x,\sigma) \;=\; \frac{1}{\sigma\sqrt{2\pi}}\,\exp\!\left(-\frac{x^2}{2\sigma^2}\right). \tag{9}$$

Standard deviation of g_{ph3}: σ.
Default: `sigmas=None` ⇒ tetrahedron method (no broadening, exact integration over the BZ within the discretized mesh).

The cross-code finding. kaldo and phono3py both expose a parameter named `sigma`, but the parameter's meaning is structurally different: Eq. (8) and Eq. (9) parameterize the Gaussian through different conventions, and the two conventions are related by $\sigma_{\mathrm{kaldo}} = \sqrt{2}\,\sigma_{\mathrm{ph3}}$ for matched standard deviation. With identical numerical σ, kaldo's effective broadening is $1/\sqrt{2}$ of phono3py's: kaldo's δ-replacement is narrower, fewer three-phonon resonances are captured, and κ comes out larger.

We measured this directly. In the silicon-Tersoff experiment with the 4×4×4 FC2 supercell, the 3×3×3 FC3 supercell, and an 8×8×8 q-mesh at 300 K, the cross-code ratios for the diagonal-averaged κ tensor are:

	Equal nominal σ		**Equal effective stdev** ($\sigma_k = \sqrt{2}\sigma_p$)	
σ_p (THz)	κ_{RTA} ratio	κ_{direct} ratio	κ_{RTA} ratio	κ_{direct} ratio
0.05	1.314	1.222	1.058	1.024
0.10	1.163	1.107	1.040	**1.004**
0.15	–	–	1.047	1.014
0.20	1.078	1.065	1.056	1.027

After the $\sqrt{2}$ correction, the direct/LBTE ratio collapses to within 3% across the whole σ range and matches to within 0.4% at $\sigma_p = 0.10$ THz. The remaining 4–6% in RTA is a separate, smaller effect (the codes implement RTA differently; the direct/LBTE solvers agree where RTA does not).

What this tells us about the substrate. The same parameter name in two codes does not guarantee the same parameter meaning. `sigma_THz` on the `compute_scattering_rates` operation must be qualified by a `sigma_convention` parameter that records which Gaussian normalization the adapter actually uses. Two adapters with the same operation name and the same `sigma_THz` but different `sigma_convention` are different operations under our parameterized-identity rule, and therefore produce formally distinct states. A third adapter (e.g., ShengBTE, when we add it) will declare its own `sigma_convention`; the substrate's job is to expose the divergence structurally rather than allow it to silently propagate as a numerical mystery.

This is the second "hidden" methodological difference the experiment surfaced. The first was the q-mesh shift convention exposed at the `Dispersion` stage; this one is exposed at `ScatteringRates`. Both are silent unless the substrate forces the comparison.

Per-mode diagnostic at matched stdev. Forcing matched effective stdev ($\sigma_p = 0.10$ THz, $\sigma_k = \sqrt{2}{\cdot}0.10$ THz), we extracted the per-mode materializations of `HeatCapacity` and `Linewidth` from each code on the same q-mesh, aligned them by q-grid permutation, and computed mode-by-mode ratios:

- **Heat capacity ratio is constant:** $c_v^{\text{kaldo}}/c_v^{\text{ph3}} = 1.6022 \times 10^{-19}$ (relative std 6×10^{-6}, i.e. to numerical precision). This is exactly the elementary charge e in J/eV, identifying the convention difference: kaldo stores c_v in J/K per mode, phono3py in eV/K per mode. Each code uses its own units consistently inside its κ formula, so the final κ in W/m K agrees, but the materialised state has two different numerical realisations.
- **Linewidth ratio has a 4π leading offset plus mode-dependent multiples of π:** the ratio $\Gamma^{\text{kaldo}}/\Gamma^{\text{ph3}}$ averages $12.48 \approx 4\pi$, but a mode-resolved breakdown is much richer. Sorting frequencies within each q-point and binning by mode index (lowest to highest):

mode	mean $\langle\omega\rangle$ (THz)	mean ratio/4π	approx convention factor
0 (TA-like)	2.6	1.12	$4\pi \times 1$
1 (TA-like)	3.1	0.97	$4\pi \times 1$
2 (mid-optical)	8.5	1.80	$\sim 7\pi$
3 (high-optical)	13.7	1.99	$8\pi \times 1$
4 (TO-near-max)	15.9	0.86	$\sim 3.4\pi$
5 (LO-near-max)	16.0	0.57	$\sim 2.3\pi$

  The pattern is more structured than a simple unit-conversion: the leading 4π is the angular-vs-linear-frequency convention (kaldo's Gaussian δ-replacement integrates against ω where phono3py's integrates against $\nu = \omega/2\pi$), but the per-mode multipliers are themselves multiples of π, suggesting that the codes distribute the 3-phonon permutation-symmetry prefactor differently across modes, N-vs-U processes, and population branches.

Definitive: per-mode redistribution, not different physics. A controlled diagnostic reveals the codes agree on the *total* scattering volume to numerical precision but redistribute it across modes differently:

quantity	kaldo/4π	phono3py	ratio
$\sum_{\mathbf{q},\nu} \Gamma_{\mathbf{q}\nu}$ (total)	205.19	205.09	1.0005
$\sum_{\nu} \Gamma_{\mathbf{q}\nu}$ (per-q sum)	—	—	$\langle\cdot\rangle = 0.9996$, std = 1.0%
$\Gamma_{\mathbf{q}\nu}$ (per mode within q)	—	—	$\langle\cdot\rangle = 0.99$, std = 3.4%

The codes compute the same gauge-invariant total scattering rate. They differ purely in how the contribution from each (q, q', q") triplet is assigned to "the mode at q," "the mode at q'," or "the mode at q'." The total $\sum \Gamma$ is invariant under this redistribution; the per-mode values are not.

Why this explains the RTA-vs-direct asymmetry. RTA computes κ as $\sum c_{q\nu} v_{q\nu}^2/(2\Gamma_{q\nu})$, a per-mode-weighted sum that is sensitive to which modes "own" which scattering channels. Direct/LBTE solves the full collision matrix; the off-diagonal terms capture the inter-mode redistribution implicitly, so κ_{direct} is much less sensitive to it. This predicts the asymmetry observed at all σ values:

- κ_{direct}: kaldo/phono3py ratio $\approx 1.00 \pm 0.03$ (insensitive)
- κ_{RTA}: kaldo/phono3py ratio $\approx 1.04 \pm 0.06$ (sensitive)

The 4–6% RTA residual is the redistribution effect projected onto $c \cdot v^2$, and not a deeper physical disagreement.

Located: structural prefactor difference in the Γ formula. Reading the actual kernels:

kaldo (`kaldo/phonons.py:1101`): Γ prefactor = $\pi\hbar/4 \cdot$ `GAMMA_TO_THZ` $/\,\omega$ with the $|V_3|^2$ tensor itself carrying a factor $1/(\omega'\omega'')$.

phono3py (`phono3py/phonon3/imag_self_energy.py:200`): Γ prefactor = $18\pi/(\hbar\,\mathrm{eV})^2/(2\pi\,\mathrm{THz})^2 \cdot \mathrm{eV}^2$ with $|V_3|^2$ carrying a factor $1/(8\,\omega\omega'\omega'')$.

Both formulas are nominally equivalent; the difference is in the bookkeeping. **phono3py carries an explicit factor of 18 from 3-phonon permutation symmetry; kaldo absorbs the same permutation symmetry differently through `GAMMA_TO_THZ` and an external $1/\omega$.** Two adapters that nominally compute "the same" linewidth produce numerically different values for modes where the permutation counting matters differently (e.g. degenerate triplets, N-process vs U-process splits, near-degenerate modes at zone-edge). The 4–6% RTA κ residual is the integral of these per-mode multiplicative discrepancies under the BTE solver.

The per-mode Γ variation is the source of the residual ∼5% RTA gap. Direct/LBTE absorbs more of this into the off-diagonal collision matrix and converges to within 0.4% — which explains why direct agrees better than RTA at fixed σ.

What the substrate learns from this. Two principles, larger than the σ-normalisation finding:

Principle (numerics). *Same numerical materialisation does not imply same numerical value across adapters.* `HeatCapacity`, `Linewidth`, and similar derived states carry implicit *unit conventions* and *prefactor conventions* that vary across codes. The substrate's `Materialization` type must include a `unit_convention` field on its discretisation scheme.

Principle (gauge-invariance). *Some abstract states have only their **aggregate** value invariant across adapters; per-mode values are gauge-dependent.* kaldo and phono3py both compute the same $\sum_{\mathbf{q},\nu} \Gamma_{\mathbf{q}\nu}$ to numerical precision, but the per-mode $\Gamma_{\mathbf{q}\nu}$ are mode-dependent gauge-redistributions of the same total. This is not a code bug; it is a real freedom in the bookkeeping of how a (q, q', q'') scattering event is attributed to the modes participating in it. The substrate must distinguish two kinds of materialisation observables:

gauge-invariant – κ, $\sum_{\mathbf{q},\nu} \Gamma_{\mathbf{q}\nu}$, anything contracted over the full mode-sum. These are well-defined cross-adapter and their numerical equality after unit conversion is a meaningful test.

gauge-dependent – per-mode $\Gamma_{\mathbf{q}\nu}$, per-mode lifetimes $\tau_{\mathbf{q}\nu}$, RTA mode-resolved $\kappa_{\mathbf{q}\nu}$. These vary across adapters by a redistribution that integrates to identity. Cross-adapter equality is *not* expected; comparing them tells us about the adapter's bookkeeping, not about the physics.

The `Materialization` type should therefore carry a `gauge_class` field on the abstract state (or equivalently, on the materialisation's discretisation scheme). The substrate's cross-adapter alignment machinery should treat numerical agreement on gauge-invariant materialisations as a real physics check, and numerical agreement on gauge-dependent ones as a diagnostic of bookkeeping conventions only.

This also predicts the RTA-vs-direct asymmetry: κ_{direct} is gauge-invariant (the collision matrix's off-diagonals capture the redistribution); κ_{RTA} is gauge-dependent (it weights per-mode τ by per-mode $c\cdot v^2$, so redistribution propagates into the answer). Empirically: kaldo/phono3py κ_{direct} ratio $\approx 1.00 \pm 0.03$; κ_{RTA} ratio $\approx 1.04 \pm 0.06$ across σ.

This is also a falsifiable substrate prediction: when ShengBTE is added, its `Linewidth` materialisation will exhibit (a) its own constant unit offset (likely a different multiple of π)

and (b) its own per-mode pattern reflecting its own symmetry-distribution choice. The substrate's job is to expose both — the constant factor as a unit label, the per-mode variation as a `permutation_distribution` parameter — so that cross-code comparison after adapter alignment is meaningful.

9 `ThermalConductivity` (brief)

Abstract

The lattice thermal conductivity tensor is the contracted observable

$$\kappa^{\alpha\beta}(T) = \frac{1}{V N_q} \sum_{\mathbf{q}\nu} c_{\mathbf{q}\nu}(T)\, v^{\alpha}_{\mathbf{q}\nu}\, F^{\beta}_{\mathbf{q}\nu}(T), \tag{10}$$

where c is the mode heat capacity, $\mathbf{v}$ are the group velocities, and $\mathbf{F}$ is the mean free displacement obtained from the BTE solution.

Substrate type: `THERMAL_CONDUCTIVITY`.
Operation: `contract_to_thermal_conductivity`.
Operation parameters: `bte_solver` $\in$ {`rta`, `iterative`, `direct_inverse`}.

Materialization in kaldo

`Conductivity(phonons=phonons, method='rta'|'inverse'|'sc')`.
For 4×4×4 FC2, 3×3×3 FC3, 8×8×8 mesh, $\sigma = 0.1414$ THz: $\kappa_{\mathrm{RTA}} = 17.41$, $\kappa_{\mathrm{inv}} = 24.39$ W/m K.

Materialization in phonopy / phono3py

`ph3.run_thermal_conductivity(is_LBTE=False)` (RTA) or `is_LBTE=True` (direct).
At the same discretization with $\sigma = 0.10$ THz: $\kappa_{\mathrm{RTA}} = 16.73$, $\kappa_{\mathrm{LBTE}} = 24.30$ W/m K.

Note on absolute values. These κ are well below the published Si Tersoff value (∼250 W/m K) because of mesh under-convergence at 8×8×8. Both adapters are equally under-converged here; the cross-code ratio is the substrate-relevant signal, the absolute number is not.

10 What we learned

The contact between architecture and physics surfaced six observations.

1. **The upstream stub is honest.** Both kaldo and phonopy treat the potential as an opaque label. The user supplies it externally (Tersoff parameters, DFT functional, ML weights) and neither code symbolically inspects V. Phase 1's stubbed-upstream choice matches both adapters exactly: `Potential` is a parameter state with an opaque-label materialization, and the upstream operations are recorded as provenance only.

2. **The Tersoff/Taylor mismatch is not a bug; it's a feature of the parameterized operation identity.** The same abstract derivative-extraction operation admits multiple methods (finite difference, DFPT, autodiff). Each method is a distinct operation identity. Materializations of the same `ForceConstants[order=`n`]` type produced by different methods carry different provenance and are formally distinct — which is exactly what we want for cross-method comparison.

3. **Second- and third-order force constants are independent derived states.** Different supercells, different methods, independent discretizations. The type-level `order` parameter is doing real work in the substrate; `FORCE_CONSTANTS` with no `order` parameter would be ambiguous.

4. **Structural operations (Fourier, eigendecomposition) are unambiguous.** kaldo and phonopy implement them in different files, in different languages, with different internal data layouts, but both produce the same abstract state. These operations are good places to validate the materialization-functor alignment machinery before introducing operations where the codes actually disagree.

5. **Correction operations have a uniform shape.** `enforce_acoustic_sum_rule` (and the LO–TO splitting, permutation symmetrization, and Born–Huang invariance operations that will follow) all share the pattern $\tau \to \tau$: same input and output type, distinguished only by an additional provenance entry on the output. By Decision 5 (identity = type + provenance), input and output are formally distinct states despite sharing a type. This pattern lets us factor cross-code disagreements that live in correction *variants* and *timing*, not in the operation's structural type signature.

6. **Same parameter name $\neq$ same parameter meaning.** The `compute_scattering_rates` operation in both kaldo and phono3py exposes a parameter named `sigma`, but the parameter is a different number for each: kaldo's σ and phono3py's σ correspond to different Gaussian standard deviations ($\sigma_k = \sqrt{2}\,\sigma_p$ for matched stdev, see Eqs. (8)–(9)). Naively matching numerical σ left a 7–30% κ gap; correcting for the $\sqrt{2}$ factor closes nearly all of it (down to <3% with the direct BTE solver). *The substrate must therefore record not only the operation parameter's value but its convention.* We anticipate a `sigma_convention` parameter on `compute_scattering_rates`, with kaldo, phono3py, and (when added) ShengBTE each declaring their own convention. Without it, two states with identical `(operation_name, sigma_value)` are silently inequivalent.

Architectural status. The substrate's commitments (Principles 1, 3, 5, 6, 7, 9) survive contact with the silicon-Tersoff workflow through κ. The parameterized-operation-identity rule (Decision 5.1) is doing real work: it is what lets us formally distinguish "same σ value, different σ convention" as two operations producing two different states. The operation registry will continue to grow as we discover more sub-parameters that codes treat differently — the discovery is empirical, driven by cross-code experiments like this one. Adding ShengBTE next will exercise this machinery further: ShengBTE has its own broadening defaults, its own scattering-rate normalization, and its own iterative-BTE convergence criteria. The substrate is built to surface those silently as new parameter dimensions on existing operations rather than as new operations.